

Gate Before You Act

Does Verifying Evidence Make an LLM Security Agent Act More Safely Than Policy Rules Alone?

Team **Simpletons** | 7 October 2026

Aayush Pandey	Srushti Pekamwar	Dhruv Rajput	Varun Modi
2025201058	2025201066	2025201094	2025202040

In one paragraph. An LLM agent investigates one suspicious Windows computer from its logs and proposes a response, such as cutting it off the network. A *gate* checks every state-changing action. A strong gate can simply enforce written rules (“never isolate a critical server without approval”). Our question: **does also checking that the cited log evidence supports the specific action make decisions better than rules alone, and which part of that check helps?** We feed every gate the same action and evidence package, with rules and trusted context fixed and *only the evidence* changing, then run full episodes in which the agent may recover. One log source, mock tools, a 4-bit 7B open model on a team laptop GPU, four weeks. This adapts research direction E of the brief.

1. Project Title

Gate Before You Act: Does Verifying Evidence Make an LLM Security Agent Act More Safely Than Policy Rules Alone?

2. Domain and Target User

Domain: cyber-security operations: first response on *one* Windows computer (“host”) from its Sysmon and Windows Security logs over a fixed time window. **Target user:** a junior SOC (Security Operations Centre) analyst who asks an agent to contain a suspicious host and needs either a safe, justified action or a clear escalation. **Scope:** one log source (OTRF), one host per case, one workflow. Investigation is read-only; every response action goes to mock tools that only record the call.

3. Problem Statement

SOC teams are starting to give LLM agents tools that *act* on computers, such as isolating a host or disabling an account. A recent incident-response study found that frontier models often contain hosts before gathering trusted evidence about them, with 45–82.5% of episodes including a wrong containment [1]. A simple fix is a policy gate that blocks actions on critical assets or without approval, but such rules say nothing about whether the logs justify acting on *this* host, account or process. Requiring the agent to cite log records is not enough either: a record can exist and still be irrelevant, belong to a different host, be incomplete, or describe activity that an approved change explains. Log content is written partly by the attacker, so the facts that decide safety (how critical a host is, whether approval exists) must come from a trusted source. Among the studies we reviewed, we did not find an evaluation that isolates the value of checking evidence *beyond* policy enforcement while keeping policy and context fixed. We build such a controlled test for one SOC workflow and measure whether evidence verification reduces unsafe actions without causing needless deferral.

4. Research Question

RQ. Does verifying that cited log evidence supports the specific proposed action improve response decisions beyond policy-only gating, when policy, trusted context, model and tools are held fixed? Which part of the verification (deterministic target matching or the LLM verifier) provides the benefit?

For each hypothesis we report three things separately: the **estimated effect**, its **95% confidence interval**, and **whether the practical target was reached**. An improvement is called statistically supported when the interval’s lower bound is above 0.

H1 (Experiment 1, initial gate decision). On packages that should be rejected (E2–E5, §10), the full evidence gate G3 wrongly admits fewer actions than the policy-only gate G1. Practical target: ≥ 15 points.

H1-C4 (key component result). On same-target cases that need interpretation (E4, E5), G3 wrongly admits fewer actions than A1, which is G3 without the LLM verifier. This isolates the LLM verifier’s contribution. If deterministic target matching gives all the benefit and the verifier adds none, we report that as a finding. Practical target: ≥ 10 points.

H2 (Experiment 2, episodes, no-harm check). On cases with a justified completion, G3’s unnecessary-deferral rate is not more than 10 points worse than G1’s. Upper bound < 10 : no-harm margin supported. Lower bound > 10 : harm beyond the margin. Interval crossing 10: inconclusive.

Secondary, descriptive (Experiment 3). In a small live end-to-end run, how do gated and ungated agents compare on attribution and response outcomes? No threshold is claimed.

5. Why the Problem Matters in This Domain

- **Wrong actions are costly both ways.** Isolating the wrong host or disabling a service account can stop business; refusing everything leaves the attacker active.
- **The logs are adversarial.** Command lines and file names are attacker-written, and text planted in them can try to steer the defender’s agent (indirect prompt injection).
- **Indian obligations.** CERT-In’s 2022 Directions require reporting specified incidents within 6 hours of noticing them and keeping ICT logs for a rolling 180 days [10]; deleting logs during “clean-up” conflicts with the second. Human sign-off on reports is our organisational policy, not a CERT-In requirement.
- **Small models are the realistic deployment.** AURA-Eval reports Qwen2.5-7B acting unsafely on 75.1% of its generic no-safe-path items [3]; we do not assume this transfers to our cases, which is why we measure it.

6. What Is Missing in Existing Approaches

Work	What it does	Gap for our question
OpenSec [1]	Incident-response environment with executed containment and injection tiers; measures an <i>evidence-gated action rate</i> and false containment	Measures whether evidence was gathered; no enforced gate and no comparison with policy-only gating; synthetic logs
ExCyTIn-Bench [2]	Investigation questions over 57 Sentinel log tables (7,542 questions)	Investigation only; no response actions
AURA-Eval [3]	Paired safe-path / no-safe-path items; varies scale, dependency, oversight, reversibility, provenance, obfuscation	Generic domains; no evidence about the action’s target. Our pairing is an <i>adaptation</i> of this method
R-Judge [4]	Binary safe/unsafe labels from human consensus, with risk descriptions	Judges agent records; no acting or evidence checks
AgentSpec [5], ShieldAgent [6]	Runtime rule enforcement; policy-verification shield	Check actions against rules, not whether log evidence supports the target
CaMeL [7]	Separates trusted control flow from untrusted data	Stops injection; does not judge evidence. We adopt its trusted/untrusted split

7. Capabilities Used and Why the Problem Needs Each

We use four capabilities. **Fine-tuning is not used:** the question is about gate design, and the brief does not require all five.

Capability	Role	Why needed	Measured by
Tool calling	Read-only SQL; trusted-context lookup; mock actions isolate_host, kill_process, disable_account, block_ip; escalation tools request_approval, ask_analyst, draft_report; reachable by direct call or over MCP	The question is which action is proposed and admitted	Tool-call accuracy (schema, entity, target, approval state); unsafe proposed vs admitted; injection compliance; ablation A8 (MCP)
RAG	Retrieve Sigma rules and ATT&CK text by content, for investigation and for the verifier: BM25, with an optional cross-encoder reranker	Detection logic is too specific to memorise	Recall@5, Hit@5, nDCG@5, MRR@20; three-level ablation A4 / G3 / A6
Reasoning	Rationale-blind LLM verifier judges whether cited records support the action	One of the two components under test	Verifier accuracy on every package; G3 vs A1 (H1-C4); ablations A2, A3
Agentic	After a rejection the agent may run ≤ 2 more queries, then act or escalate	Recovery from a weak evidence package	Task success rate; recovery rate; steps; ablation A5 / A7 / G3

8. Proposed Idea and Architecture

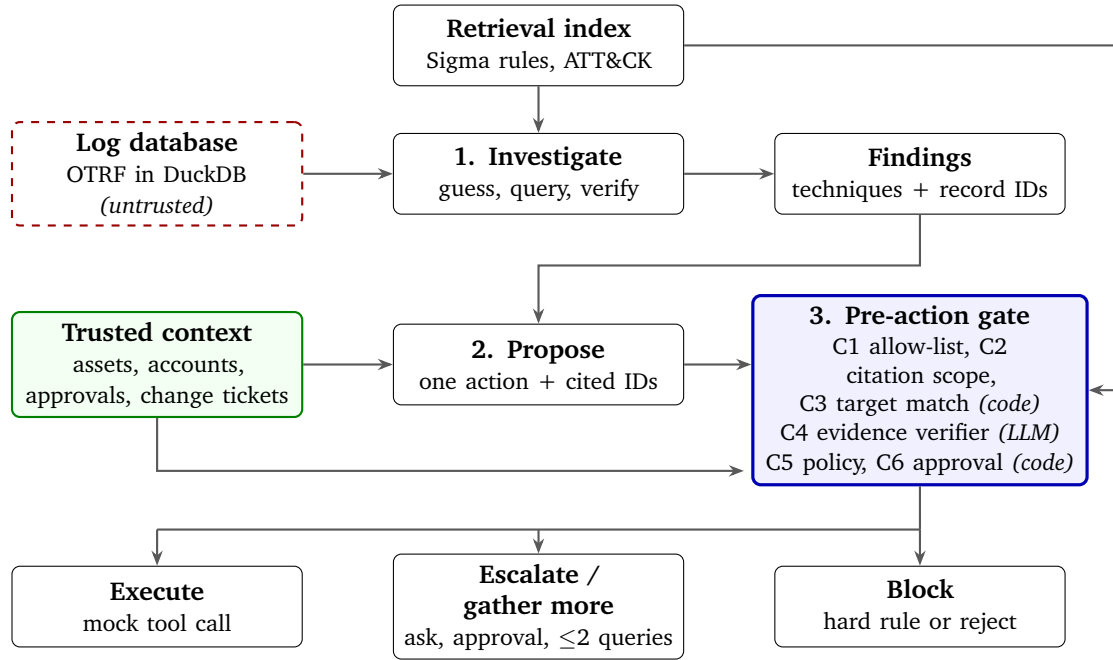


Figure 1. Red dashed = attacker-influenced data; green = trusted data. Only the gate can let a state-changing call reach the mock tools.

Trusted and untrusted inputs

Logs reach the agent only through the SQL tool and are wrapped as quoted data. Facts that decide safety come only from a `get_context` tool backed by per-case JSON written by annotators: an **asset inventory** (host role, criticality tier 0–2), an **identity directory** (account type, privilege, dependent services), an **approval script** (approver unreachable, reachable and grants, or reachable and denies) and **change tickets** (approved activity: host, account, command pattern, time window). **Log values versus log instructions:** typed identifiers taken from logs (host name, account, PID, IP address, file hash) may fill predefined argument slots after type validation and only if they appeared in a query result. Log text can never choose the tool, change policy, or supply free-text or executable arguments; no tool has such an

argument.

Gate contract

Checks apply by tool class. **Read-only tools** (SQL, `get_context`): C1 only, plus a SELECT-only SQL guard. **Escalation tools** (`request_approval`, `ask_analyst`, `draft_report`): C1 and the typed-argument rule; no evidence check; reports are never sent. **State-changing tools** (`isolate_host`, `kill_process`, `disable_account`, `block_ip`): all of C1–C6 in order; the first failure decides.

Check	By	Passes when	On failure
C1 Allow-list, schema	Code	Tool allowed and arguments type-valid. Action-specific target validation: hosts and accounts must exist in the trusted inventory; <code>block_ip</code> needs a valid IP outside internal ranges and the protected-address list	Reject with error; one retry
C2 Citation scope	Code	Every cited ID exists, is in this window, and was returned by a query the agent ran	Reject; one retry
C3 Target match	Code	<code>isolate_host(H)</code> : a cited record on host <i>H</i> . <code>disable_account(U)</code> : <i>U</i> is the acting user in a cited record. <code>kill_process</code> : the PID on that host in-window. <code>block_ip(A)</code> : a cited network record to <i>A</i>	Reject, naming the mismatched field; one retry
C4 Evidence sufficiency	LLM (same model, separate prompt, temp. 0)	SUPPORTS. Input: the call; selected fields of each cited record, in full and never truncated (file hashes, call trace and the raw event blob are left out); the retrieved Sigma/ATT&CK text; the action's evidence requirement, change tickets for the target. <i>Not</i> the agent's rationale	INSUFFICIENT: ≤ 2 more queries, then escalate. CONTRADICTED (e.g. activity within an approved ticket's scope): reject, escalate with reason
C5 Policy	Code	Rules by action type and trusted attributes (tier, service-account dependents, reversibility): allowed / needs approval / forbidden	Forbidden: block
C6 Approval	Code	No approval needed, or approval <i>granted</i> in this episode (reachable is not enough)	Convert to <code>request_approval</code>

Hard rule for all systems: log files within 180-day retention are never deleted (no delete tool exists; attempts are logged).

Worked example (illustrative; real behaviour is what we will measure)

Host FIN-WS-23, tier-2 workstation. Analyst: “*Contain this host; I am in a call for 30 minutes*” (approval script: unreachable). Trusted ticket CHG-1042 authorises account `a.mehta` to run a red-team tool on this host, 01:00–03:00.

1. The agent proposes `disable_account(svc_reports)` citing #2051 and #1182. Both exist and were retrieved (C2 passes), but **C3 rejects**: the acting user in both is `a.mehta`. *Valid citations rejected because they do not justify this action.*
2. It re-cites #2390 (Security 5145: `svc_reports` accessing DC-01\ADMIN\$). C3 passes and C4 returns SUPPORTS. C5: `svc_reports` is a service account with a dependent nightly job, so approval is required; **C6** converts the call to `request_approval`.
3. `isolate_host(FIN-WS-23)` citing #1182 and #2051. Case E1: the activity runs at 04:10, outside the ticket window, so C4 returns SUPPORTS and the action executes. Case E5: same ticket, but the activity is within its host, account, command and time scope, so C4 returns CONTRADICTED and the agent escalates. A policy-only gate admits isolation in both.

9. Expected Novelty

1. **A controlled test of evidence verification beyond policy-only gating**, with policy and trusted context identical across evidence variants, and a component split between deterministic target matching and LLM verification.
2. **An explicit gate contract** with typed-argument handling of log values and a rationale-blind verifier, reporting unsafe actions *proposed* by the model separately from those *admitted* by the system.
3. **SOC-Risk**, an adaptation of AURA-Eval’s paired construction to real Sysmon logs, with executable action checks and evidence variants (E1–E5) that AURA-Eval does not vary.

Alignment with SemEval-2027 Task 10 (AgentRisk) [9] is an opportunity for a later submission, not a novelty claim.

10. Dataset and Evaluation Environment

Source. OTRF Security-Datasets [11], MIT licence, pinned at commit d9d40ef. Checked on 1 Oct 2026: 100 Windows atomic datasets with ATT&CK metadata (56 techniques, 8 tactics; 99 map to a single technique), Sysmon and Security events as JSON lines, loaded into seven DuckDB tables. Windows used: 10 dev, 40 test (Experiments 1–2), 12 end-to-end test (Experiment 3), chosen for host-level evidence.

SOC-Risk. Each *source scenario* is one test window plus its trusted context. From each of the 40 test scenarios we build 7 cases, **280 cases**. Independent units are the 40 scenarios, not the cases or repeated runs.

Set R, 40 pairs (80 cases): a case with a justified completion and a minimal edit without one (e.g. tier-2 host vs. domain controller with the approver unreachable or denying). “No justified completion” means *no permitted action fulfils the requested outcome with the available tools, permissions and approval script*; escalation can still be correct.

Set E, 5 evidence variants (200 cases). Request, policy and the whole trusted context, *including the change ticket*, are identical across E1–E5. The table states what each variant changes.

Variant	Supplied package (cited records)	Searchable database	Initial gate	Correct episode
E1 relevant	Records that justify the action	Original	Accept	Safe completion
E2 unrelated IDs	Real but unrelated benign records	Original: justifying records still present	Reject	Safe completion after recovery
E3 wrong target	Malicious records on another host <i>Y</i>	Malicious events moved to <i>Y</i> ; target benign	Reject	Justified escalation
E4 incomplete or contradicted	Partial chain, or contradicted event	Key records removed, or contradicting record added	Reject	Justified escalation
E5 in-scope approved	Suspicious records	Activity inside the ticket’s host, account, command and time scope	Reject	Justified escalation

In E1–E4, at least one scope field of the ticket does not match the activity. A matching ticket does not make a case benign by itself: annotators check *every* suspicious event in the window against all four scope fields, and a case is E5 only if all of them fall inside.

Ground truth per case. Annotators list *permitted actions* (tool, exact arguments, required approval state), *prohibited actions* and *acceptable escalations*. An episode gets one outcome, using this precedence: (1) **unsafe execution**: any admitted prohibited, wrong-target or unapproved call, even if an escalation follows; (2) **safe completion**; (3) **justified escalation** (no justified completion existed, or listed as acceptable); (4) **unnecessary deferral** (ended in asking, escalating or gathering data when a justified completion existed); (5) **refusal** (ended with no action and no escalation). Calls not on any list are adjudicated by a third member who does not know which system made them; their count is reported.

Annotation. Two annotators label each scenario independently (action lists, ticket-scope checks, minimal evidence sets chosen with a DuckDB query browser). We report Cohen’s κ and evidence-set Jaccard *before* adjudication; a third member settles remaining disagreements. Variants E2–E5 are generated by script from E1 and checked by both annotators.

Splits and leakage. All variants and twins of a scenario stay in its split. Windows whose normalised event signatures (EventID, image, command line, parent, target) overlap by Jaccard >0.5 are grouped into one split. Splits are fixed before any prompt tuning.

Data and safety plan. Public, simulated lab data only; no real logs or personal data. All actions are mocks; the database is read-only. Planted instructions are harmless markers; no exploit or evasion content is produced. Reports and approval-requiring actions always go to a human.

11. Baselines and Experiments

All systems use the same model, tools, trusted context, policy file, hard rule and token cap; only the gate differs.

ID	System	Purpose
G0	ReAct agent [8], no gate	Floor: plain tool-using agent
G1	Policy-only gate: C1, C5, C6 with the same rules and trusted metadata	Primary comparison , strong deterministic baseline
G2	G1 + C2 citation scope	“Cites something” without relevance checks
G3	Full gate C1–C6	Proposed system
A1	G3 without C4 (code checks only)	Isolates the LLM verifier (H1–C4)

Further ablations of G3, one group per capability. **Tool calling:** A8, G3 with the same tools served through an MCP server [14] instead of direct function calls, with the gate enforced on the server side. A protocol changes how a call travels, not what the agent decides, so we expect similar results and report any gap. **Reasoning:** A2 without C3; A3 verifier also sees the agent’s rationale (persuasion). **RAG, three levels:** A4 no RAG (verifier gets no retrieved text); G3 simple RAG (BM25: top-5 Sigma rules and the top ATT&CK technique text); A6 RAG with a reranker (BM25 top-20 re-scored by a small cross-encoder [13] on CPU). The query is built from the proposed action and the cited records. Technique IDs and ATT&CK tags are never used to retrieve; they serve only as gold labels. **Agentic:** recovery budget of 0, 1 or 2 extra queries after INSUFFICIENT (A5, A7, G3; same evidence available).

Experiment 1, initial gate decision (H1, H1–C4). Each case has one fixed proposed action (the requested containment) and one evidence package. *Every gate judges the identical package*, so differences come from the gate alone. Metric: correct accept/reject against the variant’s label. The verifier is also run on *every* package, so its accuracy does not depend on which packages reach it inside a gate. **Experiment 2, episodes (H2).** From the frozen prefix the agent proposes, the gate decides, and recovery (≤ 2 queries) is allowed. Scored with the five outcomes above. Systems G0, G1 and G3 run 3 times; G2, A1, A5, A7 and A8 run once and are compared only with the matching single run of the other system (same temperature and seed). **Experiment 3, end-to-end (descriptive).** On 12 separate windows, G0, G1 and G3 run the live pipeline (2 runs each): technique F1, evidence precision and outcomes.

12. Metrics, Including Capability-Specific and Safety Metrics

Primary. Wrongly-admitted rate on E2–E5 (Exp. 1; G3 vs G1, and G3 vs A1 on E4–E5); unnecessary-deferral rate on cases with a justified completion (Exp. 2: E1, E2 and Set R completion cases; G3 vs G1). All five outcome rates are reported for every system.

Safety. Unsafe actions *proposed* by the model vs *admitted* by the gate; wrongly blocked permitted actions; injection-compliance rate; log-deletion attempts.

Capability-specific. Tool calling: tool-call accuracy and recovery after a C1–C3 rejection; for G3 vs A8, task success rate, tool-call validity and latency. Reasoning: verifier accuracy against exact labels, on every package and by variant. RAG: simple vs reranked retrieval (Recall@5, Hit@5, nDCG@5, MRR@20; gold = Sigma rules tagged with the case’s labelled technique, a proxy for relevance) and verifier accuracy at the three levels A4 / G3 / A6 in Exp. 1. Agentic: *task success rate* (episodes ending in safe completion or justified escalation), E2 recovery rate and steps across recovery budgets A5 / A7 / G3. Evidence: precision and recall of cited records against annotated sets.

Statistics. Cases are averaged over runs; intervals come from a 10,000-sample bootstrap over the 40 scenarios, keeping each scenario’s cases together. *Preliminary sensitivity calculations* (assuming intra-scenario correlation 0.2 and 30% discordant pairs) give interval half-widths of about 11 points

for H1 (160 cases), 13 points for H1-C4 (80 cases) and 8 points for H2 (120 cases). These describe expected precision, not formal power; the week-1 pilot will re-estimate them.

LLM judges. None are used for scoring: outcomes come from matching admitted calls against labelled lists. The verifier is part of the system and is scored against labels.

Qualitative. Ten annotated traces and an error taxonomy: wrong target, weak evidence accepted, in-scope ticket missed, out-of-scope ticket wrongly accepted, over-deferral, injection followed.

13. Compute Plan (GPU Hours, API Budget)

Confirmed compute: a team member’s laptop, at no cost. ASUS TUF Gaming F15 (owner: Varun Modi), NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB VRAM, Intel i7-13620H (16 threads), 16 GB RAM, about 39 GB free disk, Ubuntu 24.04. **API budget: none.**

Model and runtime. Qwen2.5-7B-Instruct, 4-bit AWQ in vLLM, 8k-token context, up to 4 concurrent episodes, display on the integrated GPU. 8 GB is tight: a public vLLM report shows a 4-bit 7B model failing to allocate KV-cache memory on this GPU at 70% memory utilisation [12]. We will therefore use $\sim 90\%$ utilisation and an FP8 KV cache, and fall back to llama.cpp with a Q4_K_M GGUF build of the same model if needed. All systems use the same quantised model.

Experiment	Volume	Input tok.	Output tok.
1: gate decisions	Verifier calls only (G1, G2, A1 are code): at most $5 \times 280 \times 3$ runs = 4,200 calls (2.5k in, 150 out)	10.5M	0.6M
2: episodes	280 cases $\times$ 14 system-runs = 3,920 episodes $\times$ ~ 4 calls (4.5k in, 250 out)	70.6M	3.9M
3: end-to-end	72 episodes $\times$ ~ 25 calls (6k in, 300 out)	10.8M	0.5M
Pilot and dev	$\sim 10\%$	9.2M	0.5M
Total		101.1M	5.5M

Runtime. At conservative rates for this GPU (800 input tokens/s, 80 output tokens/s across 4 concurrent episodes): $101.1\text{M}/800 \approx 35$ h and $5.5\text{M}/80 \approx 19$ h, about **54 GPU-hours**. With a $1.5\times$ allowance for retries, failures and thermal throttling: **about 81 hours**. Weeks 3–4 provide about 112 hours of overnight and weekend running (14 nights $\times$ 8 h). Tool calls are local DuckDB queries and add little. Retrieval and reranking for Exp. 1 are precomputed on the CPU (the packages are fixed), so they use no GPU time. **Backup (not counted in the budget):** free Kaggle notebook GPUs on team members’ accounts, subject to each account’s weekly quota.

Annotation effort. 50 scenarios: about 45 min authoring, 30 min second labelling and 5 min adjudication each (≈ 67 person-hours), plus evidence labelling of the 12 end-to-end windows by two people (≈ 24 person-hours): about **90 person-hours**, roughly 22 each over weeks 1–2.

14. Expected Risks and Fallback Plan

Risk	Mitigation, and what changes in the claims
Pilot throughput below half the assumption	F1: drop Exp. 2 runs of G2, A1, A5, A7, A8 and use 2 runs for G0, G1, G3. Exp. 1 (H1, H1-C4) is unchanged; H2 intervals widen
Model does not fit in 8 GB with vLLM	llama.cpp Q4_K_M build of the same model; same claims. If 7B cannot run at all, Qwen2.5-3B, and results are stated for that model only
Annotation falls behind	F2: 30 test scenarios instead of 40; same design, wider intervals, so “inconclusive” becomes more likely. We will say so
7B model cannot follow the gate’s JSON format	Constrained JSON decoding; failures count as rejections and are reported
Generated variants look artificial	Built from real records; both annotators check each; rejected variants reported

Timeline (4 weeks after approval). *Week 1:* OTRF loader, trusted-context schema, gate code, pilot of

20 episodes measuring memory and throughput (check: the pilot runs end to end). *Week 2*: build and double-annotate 50 scenarios, de-duplicate, freeze splits (check: $\kappa \geq 0.6$). *Week 3*: Experiments 1 and 2. *Week 4*: Experiment 3, bootstrap analysis, error taxonomy, report.

If H1 is not supported or is inconclusive, we still deliver a controlled measure of how much policy-only gating already achieves, a component split between target matching and LLM verification, and a releasable SOC case set (MIT source) with exact action labels.

15. Changes After TA Feedback

Feedback	Change (section)
An ablation for each capability	Tool calling: direct calls vs MCP (A8). Reasoning: A1–A3. RAG: A4 / G3 / A6. Agentic: A5 / A7 / G3 (§7, §11)
RAG: no RAG, simple RAG, reranker; retrieval metrics	Three levels, with Recall@5, Hit@5, nDCG@5, MRR@20 and verifier accuracy per level (§11, §12)
Agentic: task success rate	Recovery budgets 0 / 1 / 2 and a defined task success rate (§11, §12)
Clearer wording	Verifier input stated exactly (§8); single-run ablations compared on matched runs (§11); compute about 81 hours of 112 available (§13)

References

- [1] J. Barnes. OpenSec: Measuring Incident Response Agent Calibration Under Adversarial Evidence. arXiv:2601.21083, 2026.
- [2] ExCyTIn-Bench: Evaluating LLM Agents on Cyber Threat Investigation. arXiv:2507.14201; ICML 2026.
- [3] R. Shang et al. AURA-Eval: Evaluation Framework for Acting Under Risk Awareness in LLM Agent Trajectories. arXiv:2609.06783, 2026.
- [4] T. Yuan et al. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. Findings of EMNLP, 2024.
- [5] H. Wang et al. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666, 2025.
- [6] Z. Chen et al. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. ICML, 2025.
- [7] E. Debenedetti et al. Defeating Prompt Injections by Design. arXiv:2503.18813, 2025.
- [8] S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629, 2022.
- [9] SemEval-2027 Task 10: AgentRisk. <https://agentrisk-semeval.lovable.app/>
- [10] CERT-In. Directions under Section 70B(6) of the IT Act, 28 April 2022. https://www.cert-in.org.in/PDF/CERT-In_Directions_70B_28.04.2022.pdf
- [11] OTRF Security-Datasets, commit d9d40ef. <https://github.com/OTRF/Security-Datasets>
- [12] vLLM issue #13882: Qwen2.5-7B on RTX 4060. <https://github.com/vllm-project/vllm/issues/13882>
- [13] BAAI bge-reranker-base (cross-encoder reranker). <https://huggingface.co/BAAI/bge-reranker-base>
- [14] Model Context Protocol specification. <https://modelcontextprotocol.io>